RELEASE ENGINEERING · WINDOWS CLIENTS

Windows Code Signing Playbook

How signing a Windows executable proves who built it, how Windows checks that proof, what certificate expiry breaks, and how to sign from GitLab CI with the key held in an HSM.

What it proves · How Windows checks · Expiry · Getting & monitoring · What clients see · HSM + GitLab CI

IN SHORT

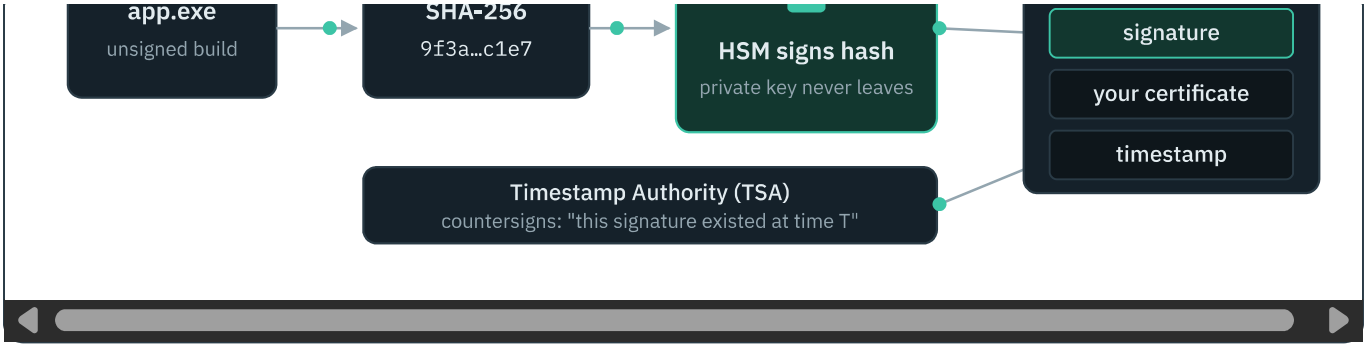
- A signature proves **who** published the file and that it is **unchanged**. It does not prove the code is safe.
- Always **timestamp** signatures. Then binaries stay valid after the certificate expires.
- Certificates now last at most about 15 months, so renewal is routine. Make the pipeline refuse to sign with an expired certificate.
- Since June 2023, publicly trusted signing keys **must** be kept in certified hardware. The CI job asks the HSM to sign; it never holds the key.
- In managed corporate environments, unsigned or badly signed binaries are usually blocked outright.

---

01 What a code signing certificate proves

Windows signatures use Microsoft's **Authenticode** format. The pipeline hashes the executable, the private key signs that hash, and the signature, your certificate and a timestamp are embedded in the file. The certificate comes from a public Certificate Authority, which checked your organization's legal identity before issuing it.





The certificate ties your public key to your verified company name. Anyone can check the signature with the public key, but only the holder of the private key could have made it.

ASSURANCE	WHAT IT MEANS FOR THE CLIENT
Authenticity	The file was signed by whoever controls the private key, and the CA vouches that this is "Your Company Ltd".
Integrity	No byte changed after signing. Changing even one byte changes the hash, and the signature fails.
Accountability	A real, vetted organization stands behind the binary. Its certificate can be revoked if it is misused.
Not covered	Whether the code is safe or bug-free. Everything depends on the private key staying secret.

02 How Windows checks the signature

When the file is opened, installed or inspected, Windows checks it through `WinVerifyTrust`. Every check must pass before Windows shows a verified publisher.

Checks, in order

1. **Hash matches**
   Windows recalculates the file's SHA-256 hash and compares it with the signed one.
2. **Chain reaches a trusted root**
   Your certificate → intermediate CA → a root in the machine's Trusted Root store.
3. **Certificate is for code signing**
   Its permitted use (Extended Key Usage)

Trust chain

Root CA
Shipped by Microsoft's Root Certificate Program, or added by the client's IT through Group Policy

`signs`

Intermediate CA
e.g. the CA's code signing issuer

its permitted use (Extended Key Usage) includes code signing, `1.3.6.1.5.5.7.3.3`.

4 **Valid at signing time**
Checked against the timestamp. Without one, Windows uses today's date.

5 **Not revoked**
Windows checks the CA's revocation lists (CRL or OCSP).

e.g. the CA's code signing issuer

signs

Your Company Ltd
Code signing certificate; its key is held in your HSM

Then each layer applies its own policy

UAC prompt

Shows "Verified publisher: Your Company" for a valid signature. Otherwise it shows "Unknown publisher" in a yellow prompt.

SmartScreen

Checks downloaded files. Its decision is based on reputation, which builds up against your publisher identity over time. Since 2024, EV certificates no longer get instant reputation.

App Control (WDAC), AppLocker, Smart App Control

Corporate rules such as "allow only binaries signed by publisher X". This is how your clients' IT teams will allow-list you.

Defender and EDR

Take the signature and reputation into account when deciding whether to flag or quarantine a file.

```
Get-AuthenticodeSignature .\app.exe | Format-List *
signtool verify /pa /v app.exe
```

03 What happens when the certificate expires

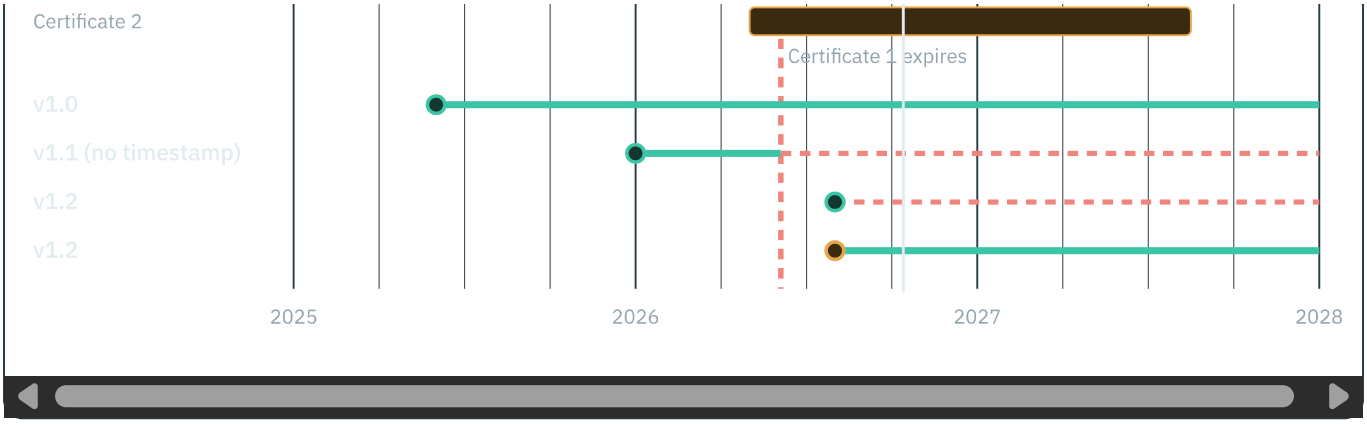
It depends on whether the signature was **timestamped**. A timestamp from a trusted Timestamp Authority proves the file was signed while the certificate was valid, so the signature stays valid after expiry. Without one, the signature fails the day the certificate expires. Drag the date to see how four releases behave.

Date on the client machine — Oct 2026 (today)

Play

Certificate 1





v1.0
Signed Jun 2025 · Certificate 1 · timestamped
Valid

v1.1
Signed Jan 2026 · Certificate 1 · no timestamp
Invalid: expired, no timestamp

v1.2 · old certificate
Signed Aug 2026 · Certificate 1 · timestamped
Invalid: certificate had expired

v1.2 · renewed certificate
Signed Aug 2026 · Certificate 2 · timestamped
Valid

Example dates. Certificate 1 runs from Mar 2025 to Jun 2026 (460 days). Certificate 2 is a renewal issued a month before Certificate 1 expires. Green lines mark periods when the signature is valid; red dashes mark periods when it is not.

- **Revocation is different from expiry.** If a certificate is revoked, signatures timestamped after the revocation date fail. If the key was compromised, the CA may backdate the revocation, which also breaks older releases.
- **Expired means you cannot sign.** No valid new signature can be made with an expired certificate. You need the renewed one.
- **Use SHA-256 throughout:** `signtool sign /fd sha256 /tr http://timestamp.digicert.com /td sha256 …`

04 Getting a certificate and watching its expiry

Fits external clients
Public CA

Simplest for CI
Microsoft

Internal only
Enterprise CA (AD CS)

DigiCert, Sectigo, GlobalSign, SSL.com. **OV** (Organization Validated) is standard; **EV** (Extended Validation) has stricter vetting, and some customers' procurement teams still ask for it. Expect the CA to check business registration and D-U-N-S, and to call back a verified phone number.

Trusted Signing

Being renamed Azure Artifact Signing. Microsoft verifies your organization and holds the keys in its own HSMs. Certificates last about 3 days and are issued automatically, so there is almost no expiry to manage. Only organizations in certain countries are eligible, so check yours first.

Only trusted on machines that trust your internal root. Your clients' machines don't, so this does not suit software you ship to them.

Making sure an expired certificate is never used

- **Gate in the pipeline.** Before signing, read the certificate's expiry date and fail the job if it has passed or is within N days. After signing, verify the signature and confirm a timestamp is present.
- **Scheduled GitLab pipeline.** Run the same check daily and alert at 60, 30 and 7 days by email or Slack, or by opening an issue.
- **Platform events.** Azure Key Vault raises a "certificate near expiry" event, and AWS and GCP offer similar alerts. Send CA renewal emails to a shared mailbox, not one person's inbox.
- **Ownership.** Track the certificate in your inventory or lifecycle tool (Venafi/CyberArk, Keyfactor, DigiCert Trust Lifecycle Manager). Name an owner and start renewal 4 to 6 weeks ahead, because the CA has to re-validate your organization.

```
# check-cert-expiry.sh: refuse to sign within 14 days of expiry
EXP=$(az keyvault certificate show --vault-name "$KV" -n "$CERT" --query attribut
DAYS=$(( ($(date -d "$EXP" +%s) - $(date +%s)) / 86400 ))
[ "$DAYS" -lt 14 ] && { echo "Certificate expires in $DAYS days - refusing to sig
```

05 What the client sees

ARTIFACT	STANDARD WINDOWS	MANAGED CORPORATE ESTATE

Signed and timestamped; certificate expired later	Valid Verified publisher	Allowed by publisher rules
Signed without a timestamp; certificate now expired	Invalid "A required certificate is not within its validity period"	Blocked; can look like tampering
Signed with an already-expired certificate	Invalid Signature fails verification	Blocked, and likely flagged by security tools
Signed; certificate later revoked	Invalid for signatures after the revocation date	Blocked
Unsigned	Unknown publisher SmartScreen "Windows protected your PC"; Edge may warn on download	Usually blocked by App Control or Smart App Control, and rejected at security intake

06 Keeping the key in an HSM and signing from GitLab CI

An HSM is required, not optional. Since June 2023, CA/Browser Forum rules require the private key of any publicly trusted code signing certificate to be generated and stored in certified hardware (FIPS 140-2 Level 2+ or Common Criteria EAL4+). CAs no longer issue downloadable `.pfx` files. The pipeline sends the hash to the HSM and gets a signature back. The key never leaves the device.

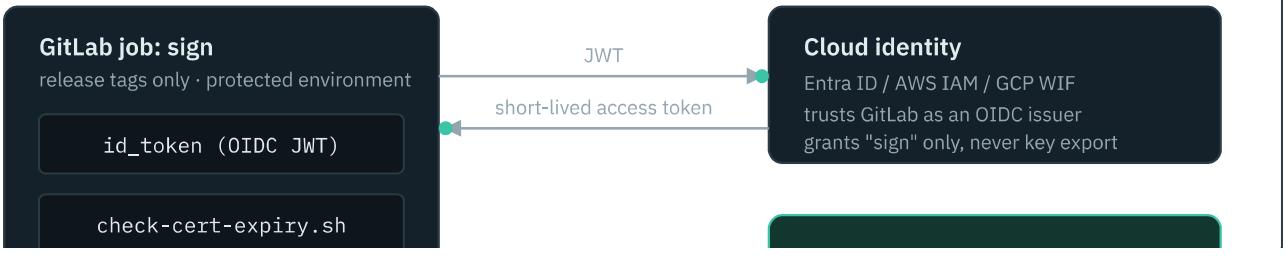
Build
compile on a protected tag

Check expiry
fail if expiring soon

Sign
OIDC → HSM, with timestamp

Verify
signature and timestamp present

Publish
HTTPS, with SHA-256 hashes





The job proves its identity with a short-lived token GitLab issues for that run. It never handles the private key, only a hash going out and a signature coming back.

Ways to connect GitLab to an HSM

OPTION	HOW GITLAB CONNECTS	TRADE-OFF
Microsoft Trusted Signing	`signtool` with Microsoft's signing plug-in, or `jsign`; authentication through GitLab OIDC → Entra ID	Simplest. No HSM or certificate renewals to manage.
CA cloud signing DigiCert KeyLocker, SSL.com eSigner, GlobalSign, Sectigo	The CA's command-line tool (e.g. `smctl`) or `jsign`, with an API key in a protected, masked CI variable	Key sits in the CA's HSM. Priced per signature or per year.
Your own cloud HSM/KMS Azure Key Vault Premium / Managed HSM, AWS CloudHSM / KMS, Google Cloud KMS (HSM)	Generate the key and certificate request in the KMS, get them certified by a CA, then sign with `jsign` or `AzureSignTool` using OIDC	Full control. Confirm first that your CA accepts that HSM's key attestation.
USB token or on-premises HSM	Attached to a dedicated, protected self-hosted Windows runner; `signtool` or PKCS#11	Works, but fragile: PIN prompts, one physical machine, hard to scale.

Example signing job (Azure Key Vault and GitLab OIDC)

`jsign` runs on Linux runners (it is Java) and supports Azure Key Vault, Trusted Signing, AWS, GCP, DigiCert and PKCS#11 HSMs. You don't need a Windows runner just to sign.

```
sign:
  stage: sign
  image: your-registry/jsign-azcli:latest
  id_tokens:
    AZURE_JWT:
      aud: api://AzureADTokenExchange
```

```
rules:
  - if: $CI_COMMIT_TAG                 # release tags only
environment: code-signing              # protected environment + approvers
script:
  - az login --service-principal -u "$AZ_CLIENT_ID" -t "$AZ_TENANT_ID" --federa
  - ./check-cert-expiry.sh             # fails if expired or close to expiry
  - TOKEN=$(az account get-access-token --resource https://vault.azure.net --qu
  - jsign --storetype AZUREKEYVAULT --keystore "$KV" --storepass "$TOKEN" --ali
      --tsaurl http://timestamp.digicert.com --tsmode RFC3161 --alg SHA-256 dis
  - osslsigncode verify dist/app.exe   # confirm signature + timestamp
artifacts:
  paths: [dist/app.exe]
```

Hardening checklist

- [ ] Sign only from protected branches or tags. Never sign in merge request pipelines, and especially not from forks